

ANSI SQL Using MySQL Exercises

Database Schema

1. Users

Column Name	Data Type	Constraints	Description
user_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique user identifier
full_name	VARCHAR(100)	NOT NULL	User's full name
email	VARCHAR(100)	UNIQUE, NOT NULL	User's email
city	VARCHAR(100)	NOT NULL	City of the user
registration_date	DATE	NOT NULL	Date when user registered

```
mysql> CREATE TABLE Users (  
->     user_id INT AUTO_INCREMENT PRIMARY KEY,  
->     full_name VARCHAR(100) NOT NULL,  
->     email VARCHAR(100) UNIQUE NOT NULL,  
->     city VARCHAR(100) NOT NULL,  
->     registration_date DATE NOT NULL  
-> );  
Query OK, 0 rows affected (0.08 sec)
```

2. Events

Column Name	Data Type	Constraints	Description
event_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique event identifier
title	VARCHAR(200)	NOT NULL	Title of the event
description	TEXT		Event description
city	VARCHAR(100)	NOT NULL	City where event is held
start_date	DATETIME	NOT NULL	Start date and time of event
end_date	DATETIME	NOT NULL	End date and time of event
status	ENUM	('upcoming','completed','cancelled')	Current status of the event
organizer_id	INT	FOREIGN KEY → Users(user_id)	User who organized the event

```
mysql> CREATE TABLE Events (
->     event_id INT AUTO_INCREMENT PRIMARY KEY,
->     title VARCHAR(200) NOT NULL,
->     description TEXT,
->     city VARCHAR(100) NOT NULL,
->     start_date DATETIME NOT NULL,
->     end_date DATETIME NOT NULL,
->     status ENUM('upcoming', 'completed', 'cancelled'),
->     organizer_id INT,
->     FOREIGN KEY (organizer_id) REFERENCES Users(user_id)
-> );
Query OK, 0 rows affected (0.06 sec)
```

3. Sessions

Column Name	Data Type	Constraints	Description
session_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique session identifier
event_id	INT	FOREIGN KEY → Events(event_id)	Associated event
title	VARCHAR(200)	NOT NULL	Title of the session
speaker_name	VARCHAR(100)	NOT NULL	Name of speaker
start_time	DATETIME	NOT NULL	Session start time
end_time	DATETIME	NOT NULL	Session end time

```
mysql> CREATE TABLE Sessions (
->     session_id INT AUTO_INCREMENT PRIMARY KEY,
->     event_id INT,
->     title VARCHAR(200) NOT NULL,
->     speaker_name VARCHAR(100) NOT NULL,
->     start_time DATETIME NOT NULL,
->     end_time DATETIME NOT NULL,
->     FOREIGN KEY (event_id) REFERENCES Events(event_id)
-> );
Query OK, 0 rows affected (0.06 sec)
```

4. Registrations

Column Name	Data Type	Constraints	Description
registration_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique registration ID
user_id	INT	FOREIGN KEY → Users(user_id)	User who registered
event_id	INT	FOREIGN KEY → Events(event_id)	Registered event
registration_date	DATE	NOT NULL	Date of registration

```
mysql> CREATE TABLE Registrations (
->     registration_id INT AUTO_INCREMENT PRIMARY KEY,
->     user_id INT,
->     event_id INT,
->     registration_date DATE NOT NULL,
->     FOREIGN KEY (user_id) REFERENCES Users(user_id),
->     FOREIGN KEY (event_id) REFERENCES Events(event_id)
-> );
Query OK, 0 rows affected (0.04 sec)
```

5. Feedback

Column Name	Data Type	Constraints	Description
feedback_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique feedback ID
user_id	INT	FOREIGN KEY → Users(user_id)	User who gave feedback
event_id	INT	FOREIGN KEY → Events(event_id)	Event for which feedback is given
rating	INT	CHECK (rating BETWEEN 1 AND 5)	Rating out of 5
comments	TEXT		Optional comments
feedback_date	DATE	NOT NULL	Date of feedback

```
mysql> CREATE TABLE Feedback (
->     feedback_id INT AUTO_INCREMENT PRIMARY KEY,
->     user_id INT,
->     event_id INT,
->     rating INT CHECK (rating BETWEEN 1 AND 5),
->     comments TEXT,
->     feedback_date DATE NOT NULL,
->     FOREIGN KEY (user_id) REFERENCES Users(user_id),
->     FOREIGN KEY (event_id) REFERENCES Events(event_id)
-> );
Query OK, 0 rows affected (0.03 sec)
```

6. Resources

Column Name	Data Type	Constraints	Description
resource_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique resource ID
event_id	INT	FOREIGN KEY → Events(event_id)	Associated event
resource_type	ENUM	('pdf','image','link')	Type of resource
resource_url	VARCHAR(255)	NOT NULL	Path or link to resource
uploaded_at	DATETIME	NOT NULL	Upload timestamp

```
mysql> CREATE TABLE Resources (
  ->     resource_id INT AUTO_INCREMENT PRIMARY KEY,
  ->     event_id INT,
  ->     resource_type ENUM('pdf', 'image', 'link'),
  ->     resource_url VARCHAR(255) NOT NULL,
  ->     uploaded_at DATETIME NOT NULL,
  ->     FOREIGN KEY (event_id) REFERENCES Events(event_id)
  -> );
Query OK, 0 rows affected (0.03 sec)
```

Sample Dataset

1. Users

user_id	full_name	email	city	registration_date
1	Alice Johnson	alice@example.com	New York	2024-12-01
2	Bob Smith	bob@example.com	Los Angeles	2024-12-05
3	Charlie Lee	charlie@example.com	Chicago	2024-12-10
4	Diana King	diana@example.com	New York	2025-01-15
5	Ethan Hunt	ethan@example.com	Los Angeles	2025-02-01

```
mysql> INSERT INTO Users (user_id, full_name, email, city, registration_date)
  -> VALUES
  -> (1, 'Alice Johnson', 'alice@example.com', 'New York', '2024-12-01'),
  -> (2, 'Bob Smith', 'bob@example.com', 'Los Angeles', '2024-12-05'),
  -> (3, 'Charlie Lee', 'charlie@example.com', 'Chicago', '2024-12-10'),
  -> (4, 'Diana King', 'diana@example.com', 'New York', '2025-01-15'),
  -> (5, 'Ethan Hunt', 'ethan@example.com', 'Los Angeles', '2025-02-01');
Query OK, 5 rows affected (0.02 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

```
mysql> select * from users;
+-----+-----+-----+-----+-----+
| user_id | full_name | email | city | registration_date |
+-----+-----+-----+-----+-----+
| 1 | Alice Johnson | alice@example.com | New York | 2024-12-01 |
| 2 | Bob Smith | bob@example.com | Los Angeles | 2024-12-05 |
| 3 | Charlie Lee | charlie@example.com | Chicago | 2024-12-10 |
| 4 | Diana King | diana@example.com | New York | 2025-01-15 |
| 5 | Ethan Hunt | ethan@example.com | Los Angeles | 2025-02-01 |
+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

2. Events

event_id	title	description	city	start_date	end_date	status	organizer_id
1	Tech Innovators Meetup	A meetup for tech enthusiasts.	New York	2025-06-10 10:00:00	2025-06-10 16:00:00	upcoming	1
2	AI & ML Conference	Conference on AI and ML advancements.	Chicago	2025-05-15 09:00:00	2025-05-15 17:00:00	completed	3
3	Frontend Development Bootcamp	Hands-on training on frontend tech.	Los Angeles	2025-07-01 10:00:00	2025-07-03 16:00:00	upcoming	2

```
mysql> INSERT INTO Events(
  -> event_id, title, description, city, start_date, end_date, status, organizer_id) values
  -> (1,
  -> 'Tech Innovators Meetup',
  -> 'A meetup for tech enthusiasts.',
  -> 'New York',
  -> '2025-06-10 10:00:00',
  -> '2025-06-10 16:00:00',
  -> 'upcoming',
  -> 1),
  -> (2,
  -> 'AI & ML Conference',
  -> 'Conference on AI and ML advancements.',
  -> 'Chicago',
  -> '2025-05-15 09:00:00',
  -> '2025-05-15 17:00:00',
  -> 'completed',
  -> 3),
  -> (3,
  -> 'Frontend Development Bootcamp',
  -> 'Hands-on training on frontend tech.',
  -> 'Los Angeles',
  -> '2025-07-01 10:00:00',
  -> '2025-07-03 16:00:00',
  -> 'upcoming',
  -> 2);
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0
```

```
mysql> select * from Events;
+-----+-----+-----+-----+-----+-----+-----+
| event_id | title | description | city | start_date | end_date | status | o |
+-----+-----+-----+-----+-----+-----+-----+
| 1 | Tech Innovators Meetup | A meetup for tech enthusiasts. | New York | 2025-06-10 10:00:00 | 2025-06-10 16:00:00 | upcoming | 1 |
| 2 | AI & ML Conference | Conference on AI and ML advancements. | Chicago | 2025-05-15 09:00:00 | 2025-05-15 17:00:00 | completed | 3 |
| 3 | Frontend Development Bootcamp | Hands-on training on frontend tech. | Los Angeles | 2025-07-01 10:00:00 | 2025-07-03 16:00:00 | upcoming | 2 |
+-----+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

3. Sessions

session_id	event_id	title	speaker_name	start_time	end_time
1	1	Opening Keynote	Dr. Tech	2025-06-10 10:00:00	2025-06-10 11:00:00
2	1	Future of Web Dev	Alice Johnson	2025-06-10 11:15:00	2025-06-10 12:30:00
3	2	AI in Healthcare	Charlie Lee	2025-05-15 09:30:00	2025-05-15 11:00:00
4	3	Intro to HTML5	Bob Smith	2025-07-01 10:00:00	2025-07-01 12:00:00

```
mysql> INSERT INTO Sessions
-> (session_id, event_id, title, speaker_name, start_time, end_time)
-> VALUES
-> (1, 1, 'Opening Keynote', 'Dr. Tech',
-> '2025-06-10 10:00:00', '2025-06-10 11:00:00'),
->
-> (2, 1, 'Future of Web Dev', 'Alice Johnson',
-> '2025-06-10 11:15:00', '2025-06-10 12:30:00'),
->
-> (3, 2, 'AI in Healthcare', 'Charlie Lee',
-> '2025-05-15 09:30:00', '2025-05-15 11:00:00'),
->
-> (4, 3, 'Intro to HTML5', 'Bob Smith',
-> '2025-07-01 10:00:00', '2025-07-01 12:00:00');
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

```
mysql> select * from sessions;
+-----+-----+-----+-----+-----+-----+
| session_id | event_id | title           | speaker_name | start_time      | end_time        |
+-----+-----+-----+-----+-----+-----+
| 1          | 1        | Opening Keynote | Dr. Tech     | 2025-06-10 10:00:00 | 2025-06-10 11:00:00 |
| 2          | 1        | Future of Web Dev | Alice Johnson | 2025-06-10 11:15:00 | 2025-06-10 12:30:00 |
| 3          | 2        | AI in Healthcare | Charlie Lee   | 2025-05-15 09:30:00 | 2025-05-15 11:00:00 |
| 4          | 3        | Intro to HTML5   | Bob Smith    | 2025-07-01 10:00:00 | 2025-07-01 12:00:00 |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

4. Registrations

registration_id	user_id	event_id	registration_date
1	1	1	2025-05-01
2	2	1	2025-05-02
3	3	2	2025-04-30
4	4	2	2025-04-28
5	5	3	2025-06-15

```
mysql> INSERT INTO Registrations
-> (registration_id, user_id, event_id, registration_date)
-> VALUES
-> (1, 1, 1, '2025-05-01'),
-> (2, 2, 1, '2025-05-02'),
-> (3, 3, 2, '2025-04-30'),
-> (4, 4, 2, '2025-04-28'),
-> (5, 5, 3, '2025-06-15');
Query OK, 5 rows affected (0.01 sec)
Records: 5 Duplicates: 0 Warnings: 0
```

```
mysql> select * from registrations;
```

registration_id	user_id	event_id	registration_date
1	1	1	2025-05-01
2	2	1	2025-05-02
3	3	2	2025-04-30
4	4	2	2025-04-28
5	5	3	2025-06-15

```
5 rows in set (0.00 sec)
```

5. Feedback

feedback_id	user_id	event_id	rating	comments	feedback_date
1	3	2	4	Great insights!	2025-05-16
2	4	2	5	Very informative.	2025-05-16
3	2	1	3	Could be better.	2025-06-11

```
mysql> insert into Feedback(feedback_id ,user_id, event_id, rating, comments, feedback_date)
-> values
-> (1, 3, 2, 4, 'Great insights!', '2025-05-16'),
-> (2, 4, 2, 5, 'Very informative.', '2025-05-16'),
-> (3, 2, 1, 3, 'Could be better.', '2025-06-11');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0
```

```
mysql> select * from feedback;
```

feedback_id	user_id	event_id	rating	comments	feedback_date
1	3	2	4	Great insights!	2025-05-16
2	4	2	5	Very informative.	2025-05-16
3	2	1	3	Could be better.	2025-06-11

```
3 rows in set (0.00 sec)
```

6. Resources

resource_id	event_id	resource_type	resource_url	uploaded_at
1	1	pdf	https://portal.com/resources/tech_meetup_agenda.pdf	2025-05-01 10:00:00
2	2	image	https://portal.com/resources/ai_poster.jpg	2025-04-20 09:00:00
3	3	link	https://portal.com/resources/html5_docs	2025-06-25

```
mysql> INSERT INTO Resources
-> (resource_id, event_id, resource_type, resource_url, uploaded_at)
-> VALUES
-> (1, 1, 'pdf',
-> 'https://portal.com/resources/tech_meetup_agenda.pdf',
-> '2025-05-01 10:00:00'),
->
-> (2, 2, 'image',
-> 'https://portal.com/resources/ai_poster.jpg',
-> '2025-04-20 09:00:00'),
->
-> (3, 3, 'link',
-> 'https://portal.com/resources/html5_docs',
-> '2025-06-25 15:00:00');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

```
mysql> select * from resources;
+-----+-----+-----+-----+-----+
| resource_id | event_id | resource_type | resource_url | uploaded_at |
+-----+-----+-----+-----+-----+
| 1 | 1 | pdf | https://portal.com/resources/tech_meetup_agenda.pdf | 2025-05-01 10:00:00 |
| 2 | 2 | image | https://portal.com/resources/ai_poster.jpg | 2025-04-20 09:00:00 |
| 3 | 3 | link | https://portal.com/resources/html5_docs | 2025-06-25 15:00:00 |
+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

Exercises

1. User Upcoming Events

Show a list of all upcoming events a user is registered for in their city, sorted by date.

```
mysql> SELECT
-> u.full_name,
-> e.title,
-> e.city,
-> e.start_date
-> FROM Users u
-> JOIN Registrations r
-> ON u.user_id = r.user_id
-> JOIN Events e
-> ON r.event_id = e.event_id
-> WHERE e.status = 'upcoming'
-> AND u.city = e.city
-> ORDER BY e.start_date;
+-----+-----+-----+-----+
| full_name | title | city | start_date |
+-----+-----+-----+-----+
| Alice Johnson | Tech Innovators Meetup | New York | 2025-06-10 10:00:00 |
| Ethan Hunt | Frontend Development Bootcamp | Los Angeles | 2025-07-01 10:00:00 |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

2. Top Rated Events

Identify events with the highest average rating, considering only those that have received at least 10 feedback submissions.

```
mysql> SELECT
->     e.event_id,
->     e.title,
->     ROUND(AVG(f.rating), 2) AS avg_rating,
->     COUNT(*) AS total_feedback
-> FROM Events e
-> JOIN Feedback f
->     ON e.event_id = f.event_id
-> GROUP BY e.event_id, e.title
-> HAVING COUNT(*) >= 10
-> ORDER BY avg_rating DESC;
Empty set (0.01 sec)
```

3. Inactive Users

Retrieve users who have not registered for any events in the last 90 days.

```
mysql> SELECT
->     u.user_id,
->     u.full_name
-> FROM Users u
-> LEFT JOIN Registrations r
->     ON u.user_id = r.user_id
-> GROUP BY u.user_id, u.full_name
-> HAVING MAX(r.registration_date) < CURDATE() - INTERVAL 90 DAY
->     OR MAX(r.registration_date) IS NULL;
+-----+-----+
| user_id | full_name |
+-----+-----+
|      1 | Alice Johnson |
|      2 | Bob Smith |
|      3 | Charlie Lee |
|      4 | Diana King |
|      5 | Ethan Hunt |
+-----+-----+
5 rows in set (0.01 sec)
```

4. Peak Session Hours

Count how many sessions are scheduled between 10 AM to 12 PM for each event.

```
mysql> SELECT
->     e.event_id,
->     e.title,
->     COUNT(s.session_id) AS sessions_between_10_and_12
-> FROM Events e
-> LEFT JOIN Sessions s
->     ON e.event_id = s.event_id
-> WHERE TIME(s.start_time) >= '10:00:00'
->     AND TIME(s.end_time) <= '12:00:00'
-> GROUP BY e.event_id, e.title;
```

event_id	title	sessions_between_10_and_12
1	Tech Innovators Meetup	1
3	Frontend Development Bootcamp	1

2 rows in set (0.00 sec)

5. Most Active Cities

List the top 5 cities with the highest number of distinct user registrations.

```
mysql> SELECT
->     u.city,
->     COUNT(DISTINCT r.user_id) AS total_users
-> FROM Users u
-> JOIN Registrations r
->     ON u.user_id = r.user_id
-> GROUP BY u.city
-> ORDER BY total_users DESC
-> LIMIT 5;
```

city	total_users
Los Angeles	2
New York	2
Chicago	1

3 rows in set (0.01 sec)

6. Event Resource Summary

Generate a report showing the number of resources (PDFs, images, links) uploaded for each event.

```
mysql> SELECT
->     e.event_id,
->     e.title,
->     SUM(CASE WHEN r.resource_type = 'pdf' THEN 1 ELSE 0 END) AS pdf_count,
->     SUM(CASE WHEN r.resource_type = 'image' THEN 1 ELSE 0 END) AS image_count,
->     SUM(CASE WHEN r.resource_type = 'link' THEN 1 ELSE 0 END) AS link_count
-> FROM Events e
-> LEFT JOIN Resources r
->     ON e.event_id = r.event_id
-> GROUP BY e.event_id, e.title;
```

event_id	title	pdf_count	image_count	link_count
1	Tech Innovators Meetup	1	0	0
2	AI & ML Conference	0	1	0
3	Frontend Development Bootcamp	0	0	1

3 rows in set (0.00 sec)

7. Low Feedback Alerts

List all users who gave feedback with a rating less than 3, along with their comments and associated event names.

```
mysql> SELECT
->     u.full_name,
->     e.title AS event_name,
->     f.rating,
->     f.comments
-> FROM Feedback f
-> JOIN Users u
->     ON f.user_id = u.user_id
-> JOIN Events e
->     ON f.event_id = e.event_id
-> WHERE f.rating < 3;
Empty set (0.00 sec)
```

8. Sessions per Upcoming Event

Display all upcoming events with the count of sessions scheduled for them.

```
mysql> SELECT
->     e.event_id,
->     e.title,
->     COUNT(s.session_id) AS session_count
-> FROM Events e
-> LEFT JOIN Sessions s
->     ON e.event_id = s.event_id
-> WHERE e.status = 'upcoming'
-> GROUP BY e.event_id, e.title;
```

event_id	title	session_count
1	Tech Innovators Meetup	2
3	Frontend Development Bootcamp	1

2 rows in set (0.00 sec)

9. Organizer Event Summary

For each event organizer, show the number of events created and their current status (upcoming, completed, cancelled).

```
mysql> SELECT
->     u.user_id,
->     u.full_name,
->     e.status,
->     COUNT(*) AS total_events
-> FROM Users u
-> JOIN Events e
->     ON u.user_id = e.organizer_id
-> GROUP BY u.user_id, u.full_name, e.status
-> ORDER BY u.user_id;
```

user_id	full_name	status	total_events
1	Alice Johnson	upcoming	1
2	Bob Smith	upcoming	1
3	Charlie Lee	completed	1

3 rows in set (0.00 sec)

10. Feedback Gap

Identify events that had registrations but received no feedback at all.

```
mysql> SELECT e.event_id, e.title
-> FROM Events e
-> JOIN Registrations r ON e.event_id = r.event_id
-> LEFT JOIN Feedback f ON e.event_id = f.event_id
-> GROUP BY e.event_id, e.title
-> HAVING COUNT(f.feedback_id) = 0;
+-----+-----+
| event_id | title |
+-----+-----+
| 3 | Frontend Development Bootcamp |
+-----+-----+
1 row in set (0.00 sec)
```

11. Daily New User Count

Find the number of users who registered each day in the last 7 days.

```
mysql> SELECT registration_date, COUNT(*) AS user_count
-> FROM Users
-> WHERE registration_date >= CURDATE() - INTERVAL 7 DAY
-> GROUP BY registration_date;
Empty set (0.00 sec)
```

12. Event with Maximum Sessions

List the event(s) with the highest number of sessions.

```
mysql> SELECT e.event_id, e.title, COUNT(*) AS session_count
-> FROM Events e
-> JOIN Sessions s ON e.event_id = s.event_id
-> GROUP BY e.event_id, e.title
-> HAVING COUNT(*) = (
->     SELECT MAX(cnt)
->     FROM (
->         SELECT COUNT(*) cnt
->         FROM Sessions
->         GROUP BY event_id
->     ) t
-> );
+-----+-----+-----+
| event_id | title | session_count |
+-----+-----+-----+
| 1 | Tech Innovators Meetup | 2 |
+-----+-----+-----+
1 row in set (0.00 sec)
```

13. Average Rating per City

Calculate the average feedback rating of events conducted in each city.

```
mysql> SELECT e.city, ROUND(AVG(f.rating),2) AS avg_rating
-> FROM Events e
-> JOIN Feedback f ON e.event_id = f.event_id
-> GROUP BY e.city;
+-----+-----+
| city      | avg_rating |
+-----+-----+
| New York  | 3.00      |
| Chicago   | 4.50      |
+-----+-----+
2 rows in set (0.00 sec)
```

14. Most Registered Events

List top 3 events based on the total number of user registrations.

```
mysql> SELECT e.event_id, e.title, COUNT(*) AS registrations
-> FROM Events e
-> JOIN Registrations r ON e.event_id = r.event_id
-> GROUP BY e.event_id, e.title
-> ORDER BY registrations DESC
-> LIMIT 3;
+-----+-----+-----+
| event_id | title                                | registrations |
+-----+-----+-----+
| 1        | Tech Innovators Meetup              | 2            |
| 2        | AI & ML Conference                  | 2            |
| 3        | Frontend Development Bootcamp       | 1            |
+-----+-----+-----+
3 rows in set (0.01 sec)
```

15. Event Session Time Conflict

Identify overlapping sessions within the same event (i.e., session start and end times that conflict).

```
mysql> SELECT
-> s1.event_id,
-> s1.session_id AS session1,
-> s2.session_id AS session2
-> FROM Sessions s1
-> JOIN Sessions s2
-> ON s1.event_id = s2.event_id
-> AND s1.session_id < s2.session_id
-> AND s1.start_time < s2.end_time
-> AND s1.end_time > s2.start_time;
Empty set (0.00 sec)
```

16. Unregistered Active Users

Find users who created an account in the last 30 days but haven't registered for any events.

```
mysql> SELECT u.user_id, u.full_name
-> FROM Users u
-> LEFT JOIN Registrations r
-> ON u.user_id = r.user_id
-> WHERE u.registration_date >= CURDATE() - INTERVAL 30 DAY
-> AND r.registration_id IS NULL;
Empty set (0.00 sec)
```

17. Multi-Session Speakers

Identify speakers who are handling more than one session across all events.

```
mysql> SELECT speaker_name, COUNT(*) AS total_sessions
-> FROM Sessions
-> GROUP BY speaker_name
-> HAVING COUNT(*) > 1;
Empty set (0.00 sec)
```

18. Resource Availability Check

List all events that do not have any resources uploaded.

```
mysql> SELECT e.event_id, e.title
-> FROM Events e
-> LEFT JOIN Resources r
-> ON e.event_id = r.event_id
-> WHERE r.resource_id IS NULL;
Empty set (0.00 sec)
```

19. Completed Events with Feedback Summary

For completed events, show total registrations and average feedback rating.

```
mysql> SELECT
-> e.event_id,
-> e.title,
-> COUNT(DISTINCT r.registration_id) AS total_registrations,
-> ROUND(AVG(f.rating),2) AS avg_rating
-> FROM Events e
-> LEFT JOIN Registrations r
-> ON e.event_id = r.event_id
-> LEFT JOIN Feedback f
-> ON e.event_id = f.event_id
-> WHERE e.status = 'completed'
-> GROUP BY e.event_id, e.title;
```

event_id	title	total_registrations	avg_rating
2	AI & ML Conference	2	4.50

1 row in set (0.00 sec)

20. User Engagement Index

For each user, calculate how many events they attended and how many feedbacks they submitted.

```
mysql> SELECT
-> u.user_id,
-> u.full_name,
-> COUNT(DISTINCT r.event_id) AS events_attended,
-> COUNT(DISTINCT f.feedback_id) AS feedbacks_given
-> FROM Users u
-> LEFT JOIN Registrations r
-> ON u.user_id = r.user_id
-> LEFT JOIN Feedback f
-> ON u.user_id = f.user_id
-> GROUP BY u.user_id, u.full_name;
```

user_id	full_name	events_attended	feedbacks_given
1	Alice Johnson	1	0
2	Bob Smith	1	1
3	Charlie Lee	1	1
4	Diana King	1	1
5	Ethan Hunt	1	0

5 rows in set (0.00 sec)

21. Top Feedback Providers

List top 5 users who have submitted the most feedback entries.

```
mysql> SELECT
-> u.user_id,
-> u.full_name,
-> COUNT(*) AS total_feedbacks
-> FROM Users u
-> JOIN Feedback f
-> ON u.user_id = f.user_id
-> GROUP BY u.user_id, u.full_name
-> ORDER BY total_feedbacks DESC
-> LIMIT 5;
```

user_id	full_name	total_feedbacks
2	Bob Smith	1
3	Charlie Lee	1
4	Diana King	1

3 rows in set (0.00 sec)

22. Duplicate Registrations Check

Detect if a user has been registered more than once for the same event.

```
mysql> SELECT
-> user_id,
-> event_id,
-> COUNT(*) AS duplicate_count
-> FROM Registrations
-> GROUP BY user_id, event_id
-> HAVING COUNT(*) > 1;
```

Empty set (0.00 sec)

23. Registration Trends

Show a month-wise registration count trend over the past 12 months.

```
mysql> SELECT
-> DATE_FORMAT(registration_date, '%Y-%m') AS month,
-> COUNT(*) AS registrations
-> FROM Registrations
-> WHERE registration_date >= CURDATE() - INTERVAL 12 MONTH
-> GROUP BY month
-> ORDER BY month;
```

Empty set (0.00 sec)

24. Average Session Duration per Event

Compute the average duration (in minutes) of sessions in each event.

```
mysql> SELECT
-> e.event_id,
-> e.title,
-> ROUND(AVG(TIMESTAMPDIFF(MINUTE,
-> s.start_time,
-> s.end_time)),2) AS avg_duration_minutes
-> FROM Events e
-> JOIN Sessions s
-> ON e.event_id = s.event_id
-> GROUP BY e.event_id, e.title;
```

event_id	title	avg_duration_minutes
1	Tech Innovators Meetup	67.50
2	AI & ML Conference	90.00
3	Frontend Development Bootcamp	120.00

3 rows in set (0.00 sec)

25. Events Without Sessions

List all events that currently have no sessions scheduled under them.

```
mysql> SELECT
-> e.event_id,
-> e.title
-> FROM Events e
-> LEFT JOIN Sessions s
-> ON e.event_id = s.event_id
-> WHERE s.session_id IS NULL;
```

Empty set (0.00 sec)

